



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente
Ingeniería en Desarrollo de Software

Asignatura: **Administración de Bases de Datos**

Unidad #5

Fecha: 20 al 26 de octubre 2025

Tema: Seguridad.

Introducción:

Este material tiene como objetivo brindarte una comprensión general sobre las técnicas de respaldo, recuperación y replicación de datos en sistemas de bases de datos, con énfasis en PostgreSQL. A lo largo del contenido, aprenderás cómo proteger la integridad de la información mediante copias de seguridad, cómo recuperar datos ante fallos del sistema, y cómo implementar replicación para lograr alta disponibilidad y tolerancia a errores. Se abordarán distintos métodos de replicación como la replicación en streaming, por bloques y mediante archivos WAL, así como la configuración de entornos primario-secundario y mecanismos de conmutación por error (failover). Este enfoque te permitirá comprender tanto el funcionamiento técnico como la importancia estratégica de estas herramientas en entornos críticos.

Sección 1: Respaldo y recuperación.

En los sistemas de información y administración de bases de datos, los mecanismos de respaldo y recuperación son fundamentales para garantizar la integridad y disponibilidad de los datos. Estas técnicas permiten restaurar información ante pérdidas, fallas técnicas o errores humanos. La práctica de respaldar datos de forma regular y la capacidad de recuperar dicha información de manera efectiva constituyen una parte esencial de la estrategia de continuidad operativa de cualquier organización.

Importancia del respaldo

El respaldo (backup) consiste en hacer copias de seguridad de los datos almacenados en un sistema informático. Estas copias permiten restaurar la información en caso de que se produzca una pérdida parcial o total debido a errores del sistema, fallas del hardware, ataques cibernéticos o desastres naturales. Desde una perspectiva técnica, el respaldo protege contra la pérdida de disponibilidad y contribuye al cumplimiento de políticas de seguridad informática.

Es posible generar periódicamente copias de respaldo y conservar una serie de versiones anteriores. Cada copia de respaldo estará identificada por tiempo y fecha, y por la última transacción incluida. Una copia de respaldo debe generarse mientras la base de datos está en reposo, ya que las actualizaciones durante el copiado pueden provocar que la copia sea inconsistente. Esto es porque una base de datos activa puede tener transacciones en curso, y capturarla en ese momento puede dejar el respaldo en un estado incompleto o corrupto.



Recuperación

La recuperación de datos (data recovery) es el proceso mediante el cual se restauran los datos o el estado operativo de un sistema después de una interrupción o error. En términos teóricos, la recuperación se basa en los principios de consistencia transaccional y uso de logs o bitácoras que permiten identificar las operaciones que fueron ejecutadas correctamente y aquellas que deben repetirse o revertirse.

A continuación, se describen los pasos que deben seguirse cuando ocurre una falla en el sistema y se desea recuperar la información:

1. Detección del error

El primer paso en cualquier proceso de recuperación es identificar que ha ocurrido una falla. Esta detección puede ser manual (cuando un usuario observa un comportamiento anómalo en el sistema) o automática (mediante herramientas de monitoreo que detectan caídas del servicio o errores de integridad). Esta fase es crítica porque, si no se detecta oportunamente el problema, puede escalar o causar mayores daños en el sistema.

El proceso de recuperación se inicia al detectar la existencia de un error. Es posible distinguir una variedad de puntos de entrada en el proceso de recuperación. Se considerarán fallas de sistemas detectadas por falta de acción del sistema, por verificaciones irrecuperables de redundancia, o por salida incorrecta observada por un usuario. Estos eventos deben activar inmediatamente los protocolos de recuperación establecidos por los administradores del sistema.

2. Determinación de la fuente del error

Una vez que el sistema ha identificado que existe un error, es necesario comprender su origen. Esta tarea, conocida como diagnóstico, es esencial para tomar decisiones informadas sobre el tipo de recuperación que se debe aplicar. Se requiere un análisis técnico detallado de los logs del sistema, el comportamiento de las transacciones recientes y el estado de los archivos.

Para decidir cuál es la mejor acción correctora, es necesario determinar la extensión del daño. Este esfuerzo está directamente relacionado con la determinación del tiempo y la causa del error. Después de una caída del sistema o cuando el procesamiento es interrumpido debido a una señal de error, es necesario analizar tanto las áreas del archivo de datos que sean sospechosas como las transacciones que no se concluyeron. Este proceso se apoya en técnicas como auditorías, verificación de integridad de datos, o comparación con respaldos anteriores.

3. Ubicación de errores secundarios

En sistemas complejos, un error inicial puede generar errores secundarios en otras partes del sistema. Por ejemplo, si un archivo de datos se corrompe, otras transacciones que lo usen también pueden verse afectadas. Es fundamental entonces realizar un rastreo completo del impacto del error para evitar recuperaciones incompletas.

Cuando se ha detectado un error que provocó una modificación inadecuada a un archivo, un rastreo a través de las listas de actividad encontrará aquellas transacciones que emplearon el bloque incorrecto. Entonces, es posible volver a introducir automáticamente el bloque correcto de las transacciones afectadas y producir resultados correctos. Si se actualizaron bloques mediante transacciones que leyeron bloques incorrectos antes de existir, es necesario restaurar aún más atrás el archivo. Esta tarea suele ser realizada por el

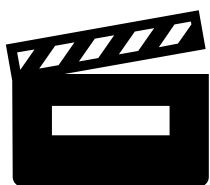


administrador de bases de datos (DBA) con ayuda de herramientas especializadas de recuperación.

4. Aplicación de correcciones

Una vez identificados todos los errores y sus efectos secundarios, se procede a la restauración del sistema. Dependiendo del grado del daño, se pueden aplicar diversas estrategias: desde restaurar un único archivo hasta ejecutar una recuperación completa del sistema a partir de un respaldo anterior.

Si la extensión del daño es limitada, puede utilizarse un proceso de volver a enrollar. Las porciones dañadas del archivo se restauran aplicando primero aquellas imágenes anteriores a los bloques en error, reemplazando después las transacciones incompletas. La salida proveniente de estas transacciones se suprime, de ser posible, para evitar duplicar resultados que previamente se hayan enviado a los usuarios. Es importante asegurar que, una vez finalizado el proceso, el sistema se encuentre en un estado consistente y confiable.



Actividad Interactiva #1

Mira el video del link presentado a continuación y escribe en las líneas de abajo un resumen de 4 líneas de lo que consideres más importante.

https://www.youtube.com/watch?v=gft_CIX9XD8&t=1276s

Sección 2: Espejo (mirroring).

El término mirroring o "espejo" es ampliamente utilizado en el ámbito del web hosting y la administración de servidores, especialmente en entornos que requieren alta disponibilidad y replicación de datos. Consiste en mantener un servidor adicional que actúa como una copia exacta del servidor principal.

El servidor espejo actúa como un "gemelo digital" del servidor principal, almacena la misma información, ejecuta los mismos servicios y está sincronizado constantemente. En caso de que el servidor principal falle, el espejo puede tomar su lugar de forma automática o manual, asegurando la continuidad de la operación sin interrupciones notables para el usuario.

Hoy en día muchas empresas, startups y agencias necesitan tener seguridad de que sus datos estarán replicados ante eventos de catástrofes naturales, cortes en la red o problemas de hardware. El uso de servidores en espejo permite cumplir con normativas de seguridad informática, brindar servicios 24/7 y proteger la reputación digital de una organización.

Dependiendo de la carga de trabajo que se tenga, así como de cuán críticos sean los datos y la demanda sobre ellos, será necesario pensar en un servidor en espejo con alta disponibilidad o bien solo en replicación pasiva.

¿Qué incluye un servidor espejo?

El servidor espejo es una copia fiel del original y contiene:



- Sistema operativo (Linux, Unix o Windows)
- Servicios del sistema
- Aplicaciones instaladas
- Configuraciones del sistema
- Archivos de usuario y de sistema
- Bases de datos (MySQL, PostgreSQL, MongoDB, entre otros)
- Correos electrónicos

Contar con toda esta información replicada permite reducir significativamente el impacto de una caída del servidor principal, que puede ser la diferencia entre una recuperación exitosa y una pérdida total de servicios e información.

¿Realmente necesito un servidor en espejo?

Para determinar si es necesario implementar un servidor espejo, es importante considerar los siguientes aspectos:

- ¿Una caída de los servicios afectaría económicamente a la organización?
- ¿Se vería comprometida la reputación de la empresa si la aplicación estuviera inactiva?
- ¿El tiempo de inactividad es tolerable (2 horas o más)?

Si la respuesta a estas preguntas indica que las consecuencias serían críticas, la implementación de un servidor en espejo es altamente recomendable. En cambio, si el impacto es bajo y se cuenta con respaldos periódicos, podría no ser necesario implementar esta solución.

Para proyectos como páginas institucionales, blogs o foros con tráfico moderado, el uso de servidores en espejo no suele ser prioritario, salvo que se gestionen grandes volúmenes de visitas (por ejemplo, más de 30 mil visitas diarias únicas).

Requisitos de Hardware y Red para un Servidor en Espejo

A continuación, se presentan los requerimientos mínimos para montar servidores espejo en sistemas Linux, los cuales ofrecen gran estabilidad para entornos de replicación:

- CPU: Intel Xeon E3-1230v3 o superior
- Memoria RAM: 8 GB
- Red interna y pública: 1 Gbps
- Distribución Linux recomendada: CentOS/RHEL, Debian o Ubuntu

Aunque es posible configurar servidores espejo con menos recursos, estos valores aseguran un buen rendimiento y estabilidad, especialmente cuando se requiere transferencia rápida de datos.

Tipos de Servidores en Espejo

1. Servidor en Espejo sin Balanceo de Carga

Este es el tipo más básico de servidor espejo, se trata de una copia idéntica del servidor principal, lista para activarse en caso de fallas críticas. Este enfoque no utiliza balanceo de carga ni alta disponibilidad, y es común en centros de datos o empresas que desean un respaldo inmediato.



En caso de falla, simplemente se asigna al servidor espejo la misma dirección IP del servidor original, y el sistema continúa funcionando. Tecnologías como DRBD y Heartbeat permiten automatizar este proceso, trasladando las IPs al servidor de respaldo cuando sea necesario.

2. Servidor Espejo con Balanceo de Carga por Software

Al integrar un balanceador de carga por software, se mejora significativamente el rendimiento del sistema. Además de la replicación de datos, se distribuye la carga entre servidores, lo que acelera las peticiones y mejora la estabilidad general de las aplicaciones.

Este balanceo puede implementarse mediante:

- Failover inteligente
- Round-robin DNS
- Software como Nginx o HAProxy

Se requiere un mínimo de dos servidores, y el balanceo puede gestionarse desde una aplicación externa, DNS o herramientas especializadas.

3. Servidor Espejo con Balanceo de Carga por Hardware

Esta solución incluye un balanceador físico dedicado, encargado de distribuir el tráfico y gestionar la alta disponibilidad. El balanceador detecta automáticamente si un servidor deja de estar disponible y redirige el tráfico hacia los servidores restantes. Cuando el servidor original vuelve a estar en línea, el tráfico se restaura de acuerdo a la configuración establecida.

Este enfoque es más costoso, pero también el más robusto. Algunas marcas populares de balanceadores de carga de hardware incluyen Cisco, Kemp Load Balancer y F5 Networks.

¿Cuánto cuesta tener servidores en espejo?

Uno de los factores determinantes a la hora de implementar servidores en espejo es el costo. Las soluciones de mirroring implican duplicar infraestructura: como mínimo se requieren dos servidores, además de un sistema de balanceo (por software o hardware) y personal especializado para la configuración y mantenimiento.

El costo de una solución con servidores espejo puede variar considerablemente dependiendo de la arquitectura y necesidades específicas:

- Dos servidores replicados: Entre \$500 y \$700 USD/mes.
- Con balanceador de carga inteligente: Entre \$1,200 y \$1,400 USD/mes.
- Con replicación de bases de datos y balanceo completo: Entre \$1,300 y \$1,500 USD/mes (incluye 4 servidores: 2 web + 2 de base de datos).

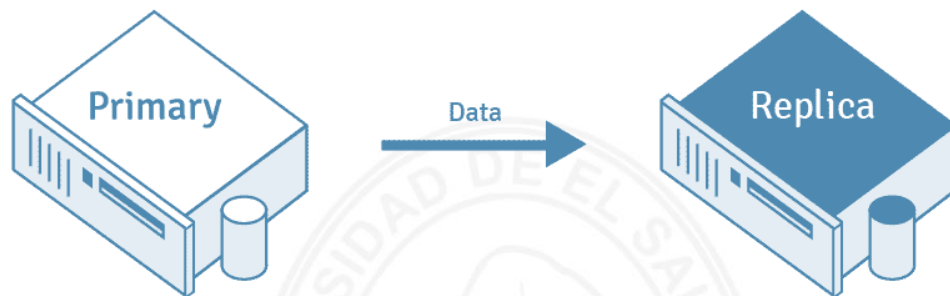
Si solo se desea replicar un sitio estático (HTML, JS, PHP sin base de datos), bastará con dos servidores. Sin embargo, si la aplicación requiere procesamiento intensivo de datos, será necesario incorporar servidores dedicados a las bases de datos, elevando los costos.



Sección 3: Réplica (replication).

La replicación de datos es una estrategia avanzada de protección y distribución de información en sistemas informáticos. Este proceso consiste en copiar de manera continua o periódica los datos de una base de datos primaria hacia una o más ubicaciones secundarias o remotas, llamadas réplicas. Esta tecnología tiene como principal objetivo mejorar la tolerancia a fallos, garantizar la alta disponibilidad de los servicios y permitir la escalabilidad de las consultas al distribuir la carga de trabajo entre varios servidores.

¿ Qué es la replicación de datos?



La replicación es un proceso automatizado mediante el cual se realizan copias continuas de los datos desde una base de datos principal (también llamada servidor primario o primary) hacia una base de datos secundaria (denominada servidor réplica o standby/replica). Esta operación puede realizarse en tiempo real (replicación en streaming) o en intervalos programados.

Uno de los objetivos principales de esta técnica es reducir el riesgo de pérdida de datos ante una falla, ya que una copia idéntica y actualizada se mantiene sincronizada en un servidor alternativo. Además, puede utilizarse para mejorar el rendimiento del sistema al permitir que las operaciones de solo lectura se distribuyan a las réplicas, liberando carga del servidor principal.

Replicación en PostgreSQL

PostgreSQL, uno de los sistemas de gestión de bases de datos relacionales más utilizados en el mundo, ofrece soporte para diversas formas de replicación. En este contexto, la replicación PostgreSQL se define como el proceso mediante el cual los datos almacenados en un servidor de base de datos principal son copiados a otro servidor que actúa como réplica.

En el modelo de replicación típico de PostgreSQL:

- Todas las operaciones de escritura (INSERT, UPDATE, DELETE) se ejecutan únicamente en el nodo primario.
- El nodo primario luego transfiere los cambios a uno o más nodos secundarios o réplicas, que actualizan sus datos para mantenerse sincronizados.
- Las consultas de solo lectura pueden ser ejecutadas en las réplicas para mejorar el rendimiento del sistema y distribuir la carga.

Este esquema garantiza que los datos estén actualizados y disponibles en múltiples ubicaciones, lo que permite mantener la operatividad del sistema incluso en caso de que el nodo principal falle.



Conmutación por error (Failover) automática

La conmutación por error, también conocida como failover, es el mecanismo que permite que una réplica asuma automáticamente el rol del servidor principal en caso de que este falle. Este procedimiento es crucial en sistemas donde la disponibilidad del servicio debe mantenerse constantemente.

En PostgreSQL, la replicación por sí sola no garantiza failover automático. Para lograrlo, se requieren herramientas externas o scripts personalizados que monitoreen la salud del servidor primario y activen la conmutación cuando sea necesario.

Ventajas del failover automático

- **Alta disponibilidad:** Reduce el tiempo de inactividad del sistema.
- **Recuperación rápida ante fallos:** Minimiza la intervención manual.
- **Continuidad operativa:** La experiencia del usuario no se ve interrumpida.
- **Protección ante desastres:** Ideal en infraestructuras críticas o aplicaciones sensibles al tiempo.

Existen soluciones como Patroni, repmgr, PgBouncer, o herramientas en la nube que permiten configurar este proceso de forma automatizada, facilitando el monitoreo y la conmutación entre nodos.

Ventajas de utilizar la replicación PostgreSQL

El uso de replicación en PostgreSQL ofrece múltiples beneficios, tanto desde la perspectiva de continuidad del negocio como del rendimiento del sistema. A continuación, se detallan los más relevantes:

- **Tolerancia a fallos:** En caso de fallo del servidor primario, una réplica puede ser promovida a nuevo primario, garantizando la disponibilidad continua de los datos y servicios sin pérdida de información significativa.
- **Mejora del rendimiento (OLTP):** En entornos de procesamiento de transacciones en línea (OLTP), la replicación permite desviar las consultas de solo lectura hacia las réplicas. Esto reduce la carga del servidor primario y mejora el tiempo de respuesta en aplicaciones altamente concurrentes.
- **Migración y despliegue sin interrupciones:** La replicación facilita la migración de datos entre servidores o entornos (por ejemplo, de desarrollo a producción) sin necesidad de detener el sistema. Esto es esencial en despliegues continuos o sistemas que no pueden detenerse.
- **Pruebas paralelas con datos reales:** Las réplicas pueden utilizarse como entornos de prueba con datos reales, lo que permite validar nuevas versiones del sistema o cambios en la base de datos sin afectar la operación en producción.

Métodos de replicación PostgreSQL

A pesar de que el modelo base de replicación en PostgreSQL es de tipo primario-secundario, existen diferentes formas de implementarla, según las necesidades de escalabilidad, rendimiento y presupuesto. Los principales métodos son:



1. Replicación en streaming

Este es el método más utilizado y consiste en transmitir los archivos de registro de escritura (WAL: Write-Ahead Logging) del nodo primario al nodo réplica en tiempo real o casi real. Esta estrategia permite mantener sincronizados los nodos con una latencia mínima.

Ventajas:

- Bajo tiempo de replicación.
- Configuración directa y bien documentada.
- Ideal para alta disponibilidad.

2. Replicación a nivel de volumen

En esta modalidad, la replicación no ocurre en la capa de base de datos, sino directamente en la capa de almacenamiento (por ejemplo, con soluciones de replicación de discos o volúmenes como DRBD). El sistema de archivos completo se sincroniza.

Ventajas:

- Independencia del motor de base de datos.
- Puede usarse con múltiples tecnologías.
- Requiere menor configuración en PostgreSQL.

Desventajas:

- Más compleja de mantener.
- Menor granularidad de control.

3. Copias de seguridad incrementales

Esta estrategia consiste en crear copias incrementales de los datos del nodo primario, y usarlas para construir réplicas o nuevos nodos en la infraestructura. Es útil para restauraciones a punto específico en el tiempo y entornos en la nube.

Ventajas:

- Ideal para respaldo histórico.
- Permite recuperación granular.
- Flexible para entornos con múltiples nodos.

Método 1: Streaming

La replicación en streaming de PostgreSQL, también conocida como replicación WAL, puede configurarse sin problemas después de instalar PostgreSQL en todos los servidores. Este enfoque de la replicación se basa en el traslado de los archivos WAL del primario a la base de datos de destino.

Puedes implementar la replicación en flujo de PostgreSQL utilizando una configuración primaria-secundaria. El servidor primario es la instancia principal que maneja la base de datos primaria y todas sus operaciones. El servidor secundario actúa como instancia complementaria y ejecuta en sí mismo todos los cambios realizados en la base de datos



primaria, generando una copia idéntica en el proceso. El primario es el servidor de lectura/escritura, mientras que el secundario es de solo lectura.

Para este método, tienes que configurar tanto el nodo primario como el nodo secundario. Las siguientes secciones detallaran los pasos necesarios para configurarlos con facilidad.

Configurar el Nodo Primario

Puedes configurar el nodo primario para la replicación en streaming llevando a cabo los siguientes pasos:

Paso 1: Inicializar la Base de Datos

Para inicializar la base de datos, puedes aprovechar el comando de utilidad initdb. A continuación, puedes crear un nuevo usuario con privilegios de replicación utilizando el siguiente comando:

```
CREATE USER 'example_username' REPLICATION LOGIN ENCRYPTED PASSWORD 'example_password';
```

El usuario tendrá que proporcionar una contraseña y un nombre de usuario para la consulta dada. La palabra clave de replicación se utiliza para dar al usuario los privilegios necesarios. Un ejemplo de consulta sería algo así:

```
CREATE USER 'rep_username' REPLICATION LOGIN ENCRYPTED PASSWORD 'rep_password';
```

Paso 2: Configurar las Propiedades de Streaming

A continuación, puedes configurar las propiedades de streaming con el archivo de configuración de PostgreSQL (postgresql.conf) que se puede modificar como :

```
wal_level = logical  
wal_log_hints = on  
max_wal_senders = 8  
max_wal_size = 1GB  
hot_standby = on
```

Información sobre los parámetros utilizados en el fragmento anterior:

1. **wal_log_hints:** Este parámetro es necesario para la función pg_rewrite, que resulta muy útil cuando el servidor secundario está desincronizado con el primario.
2. **wal_level:** Puedes utilizar este parámetro para habilitar la replicación en flujo de PostgreSQL, con valores posibles como minimal, replica, o logical.
3. **max_wal_size:** Se puede utilizar para especificar el tamaño de los archivos WAL que se pueden conservar en los archivos de registro.
4. **hot_standby:** Puedes aprovechar este parámetro para establecer una conexión de lectura con el secundario cuando está configurado como ON.



5. **max_wal_senders:** Puedes utilizar `max_wal_senders` para especificar el número máximo de conexiones concurrentes que pueden establecerse con los servidores en espera.

Paso 3: Crear una Nueva Entrada

Después de haber modificado los parámetros en el archivo `postgresql.conf`, una nueva entrada de replicación en el archivo `pg_hba.conf` puede permitir que los servidores establezcan una conexión entre sí para la replicación.

Normalmente puedes encontrar este archivo en el directorio de datos de PostgreSQL. Puedes utilizar el siguiente fragmento de código para ello:

```
host replication rep_user IPaddress md5
```

Una vez ejecutado el fragmento de código, el servidor primario permite que un usuario llamado `rep_user` se conecte y actúe como servidor en espera utilizando la IP especificada para la replicación. Por ejemplo:

```
host replication rep_user 192.168.0.22/32 md5
```

Proceso de Streaming

- El nodo secundario se conecta al primario mediante una conexión TCP/IP definida en `primary_conninfo`.
- Un proceso llamado WAL receiver se ejecuta en el secundario y escucha los archivos WAL enviados por el WAL sender del primario.
- A medida que los WAL son generados, se transmiten y aplican en tiempo real en el servidor secundario.
- Esto asegura una casi inmediata sincronización, crucial para la alta disponibilidad.

Método 2: Dispositivo de Bloque Replicado

El enfoque del dispositivo de bloque replicado depende de la duplicación de discos (también conocida como replicación de volúmenes). En este enfoque, los cambios se escriben en un volumen persistente que se replica sincrónicamente en otro volumen.

La ventaja añadida de este enfoque es su compatibilidad y la durabilidad de los datos en entornos de nube con todas las bases de datos relacionales, como PostgreSQL, MySQL y SQL Server, por nombrar algunas.

Sin embargo, el método de réplica en disco de PostgreSQL necesita que se repliquen tanto los datos del registro WAL como los de la tabla. Dado que cada escritura en la base de datos tiene que pasar por la red de forma sincrónica, no puedes permitirte perder un solo byte, ya que eso podría dejar tu base de datos en un estado corrupto.

Este método se aprovecha normalmente con Azure PostgreSQL y Amazon RDS.



Método 3: WAL

La WAL se compone de archivos de segmentos (16 MB por defecto). Cada segmento tiene uno o más registros. Un registro de secuencia de registro (LSN) es un indicador de un registro en la WAL, que permite conocer la posición/ubicación en la que se ha guardado el registro en el archivo de registro.

Un servidor en espera aprovecha los segmentos de WAL también conocidos como XLOGS en la terminología de PostgreSQL, para replicar continuamente los cambios de su servidor primario. Puedes utilizar el registro de escritura anticipada para garantizar la durabilidad y la atomicidad en un SGBD, serializando trozos de datos byte-array (cada uno con un LSN único) en un almacenamiento estable antes de que se apliquen a una base de datos.

La aplicación de una mutación a una base de datos puede llevar a varias operaciones del sistema de archivos. Una cuestión pertinente que se plantea es cómo puede una base de datos asegurar la atomicidad en caso de fallo del servidor debido a un corte de energía mientras estaba en medio de una actualización del sistema de archivos. Cuando una base de datos arranca, inicia un proceso de arranque o réplica que puede leer los segmentos de WAL disponibles y los compara con el LSN almacenado en cada página de datos (cada página de datos está marcada con el LSN del último registro de WAL que afecta a la página).

Detalles del Protocolo de Streaming WAL

Un proceso conocido como receptor de WAL, que se ejecuta en el servidor en espera, aprovecha los detalles de conexión proporcionados en el parámetro `primary_conninfo` de `recovery.conf` y se conecta al servidor primario aprovechando una conexión TCP/IP.

Para iniciar la replicación en streaming, el frontend puede enviar el parámetro de replicación dentro del mensaje de inicio. Un valor booleano de verdadero, sí, 1 u ON permite al backend saber que debe entrar en modo de replicación física `walsender`.

El emisor de la WAL es otro proceso que se ejecuta en el servidor primario y se encarga de enviar los registros de la WAL al servidor en espera a medida que se generan. El receptor de la WAL guarda los registros de la WAL en la WAL como si fueran creados por la actividad de los clientes conectados localmente.

Una vez que los registros de la WAL llegan a los archivos de segmentos de la WAL, el servidor en espera sigue reproduciendo constantemente la WAL para que el primario y el en espera estén actualizados.

Proceso de conexión y establecimiento de la replicación en streaming WAL (Write-Ahead Logging)

En PostgreSQL, específicamente entre:

- WAL Receiver en el servidor en espera (Standby Server).
- WAL Sender en el servidor primario (Primary Server).

A final de esta sección se encuentra un diagrama que es muy útil para entender cómo se inicia y autentica una conexión de replicación física en PostgreSQL, paso a paso, a nivel de protocolo. A continuación, se explica cada bloque del diagrama de manera detallada:



1. Solicitud de conexión inicial (Start up Request)

Objetivo: Establecer una conexión TCP/IP e iniciar la sesión de replicación.

Parámetros claves enviados:

- user: El nombre del usuario que se conecta, con permisos de replicación.
- database: Debe ser "replication", un valor especial en PostgreSQL.
- replication = true: Este parámetro indica al servidor que debe iniciar el proceso WAL Sender para este cliente.
- application_name: Un identificador opcional para monitorear la réplica.

Este paso inicia todo el protocolo de autenticación y replicación.

2. Solicitud de autenticación (Authentication Request)

El servidor primario solicita autenticación en formato MD5. La solicitud incluye:

- Un salt aleatorio generado por el servidor.
- La longitud del mensaje de autenticación.

Este mecanismo de autenticación es estándar para conexiones PostgreSQL que no usan certificados.

3. Respuesta de autenticación (Password Response)

El cliente (WAL Receiver) responde con:

- Un hash MD5 calculado así:
md5(md5(password) || username || salt)
- Este valor se envía al servidor para su validación.
Si el hash coincide con lo que espera el servidor, la autenticación es exitosa.

4. Confirmación de autenticación (Authentication Reply)

- El servidor responde con un mensaje de tipo Authentication OK.
- Confirma que el usuario fue autenticado correctamente.

En este punto, la conexión es segura y lista para enviar comandos de replicación.

5. Consulta de sistema: IDENTIFY_SYSTEM

El receptor de WAL envía una consulta SQL simple:

- IDENTIFY_SYSTEM;

El objetivo es obtener:

- systemid: Identificador único del sistema.
- timeline: Línea de tiempo del WAL.
- xlogpos: Posición actual en el WAL.
- dbname: Nombre de la base (si aplica).

El receptor valida que este systemid sea el mismo que el de su backup base para evitar corrupción.



6. Solicitud de replicación: START_REPLICATION SLOT

El receptor envía:

```
START_REPLICATION SLOT 'node_a_slot' 0/30000000 TIMELINE 1;
```

Esto solicita al servidor primario que comience a enviar WAL desde una posición específica (0/30000000) usando un replication slot llamado node_a_slot.

Los slots de replicación garantizan que el servidor no elimine WAL necesarios para ese receptor.

7. Respuesta del servidor: CopyBothResponse

- El servidor responde con un mensaje especial que inicia el flujo de datos WAL.
- El protocolo utilizado es una combinación de:
 - COPY (modo textual).
 - Mensajes CopyData con el contenido del WAL.

A partir de aquí, comienza el streaming continuo de los WAL desde el primario al secundario.

8. Transmisión continua de datos WAL (CopyData)

- El servidor primario comienza a enviar los archivos WAL como mensajes binarios.
- Cada mensaje lleva un identificador y 4 bytes de longitud, seguido por los datos WAL en sí.

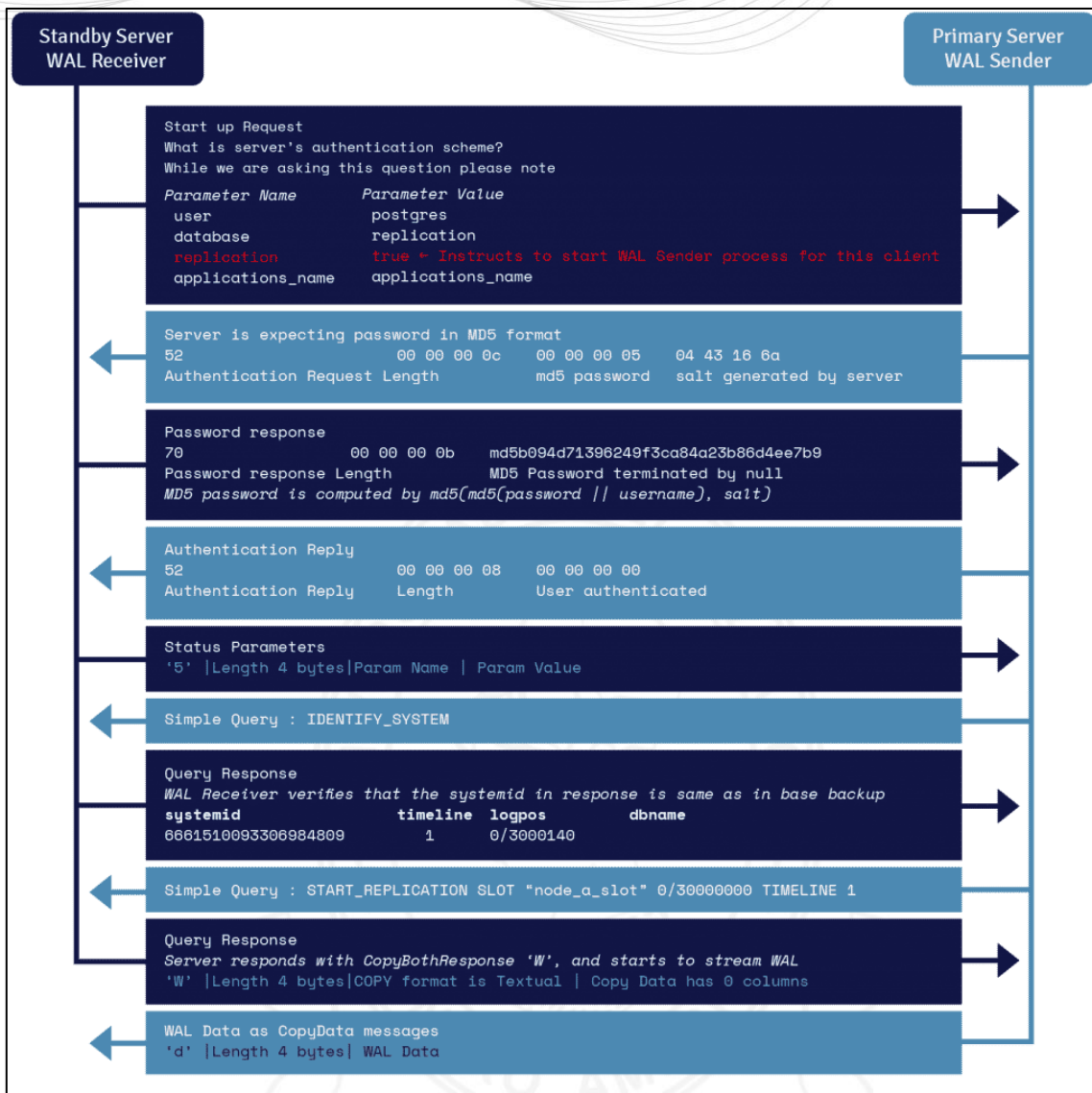
Estos datos son aplicados secuencialmente por el servidor standby para mantenerse sincronizado.

¿Por qué es importante entender esto?

Este proceso es fundamental para:

- Garantizar una replicación segura y correcta.
- Comprender cómo funciona el protocolo en bajo nivel.
- Diagnosticar problemas de replicación cuando fallan conexiones o sincronización.
- Desarrollar herramientas o sistemas que se integren con PostgreSQL como consumidores de WAL.





Configuración de la Replicación en el Servidor Primario

Realiza estos pasos una vez que hayas instalado PostgreSQL en los dos servidores primario y secundario.

1. En primer lugar, accede a la base de datos PostgreSQL con el siguiente comando:
su - postgres
2. Crea un usuario de replicación con el siguiente comando:
psql -c "CREATEUSER replication REPLICATION LOGIN CONNECTION LIMIT 1 ENCRYPTED PASSWORD'YOUR_PASSWORD';"
3. Edita pg_hba.cnf con cualquier aplicación nano en Ubuntu y añade la siguiente configuración: archivo editar comando
nano /etc/postgresql/10/main/pg_hba.conf
Para configurar el archivo, utiliza el siguiente comando:
host replication replication MasterIP/24 md5
4. Abre y edita postgresql.conf y pon la siguiente configuración en el servidor primario:




```
nano /etc/postgresql/10/main/postgresql.conf
```

Utiliza los siguientes ajustes de configuración:

```
listen_addresses = 'localhost,MasterIP'
```

```
wal_level = replica
```

```
wal_keep_segments = 64
```

```
max_wal_senders = 10
```

5. Por último, reinicia PostgreSQL en el servidor principal:

```
systemctl restart postgresql
```

Ahora has completado la configuración en el servidor primario.

Actividad Interactiva #2

Ejercicio de asociación.

A continuación, verás una lista de términos y una lista de definiciones. Asocia Cada Término con su Definición: Une cada término con la definición correcta.

Términos:

1. Replicación en streaming
2. Replicación a nivel de volumen
3. Copias de seguridad incrementales
4. Mirroring
5. Recuperación de datos
6. Respaldo (backup)



Definiciones:

- A. Técnica de protección que crea copias históricas incrementales, permitiendo restauraciones puntuales y flexibles.
- B. Consiste en mantener un servidor como copia exacta del principal, común en web hosting para alta disponibilidad.
- C. Estrategia de seguridad que genera copias de datos almacenados para prevenir pérdidas por errores, ataques o fallas.
- D. Proceso que transmite en tiempo real los archivos WAL desde el nodo primario al nodo réplica para mantenerlos sincronizados con baja latencia.
- E. Método de replicación que trabaja directamente a nivel del sistema de almacenamiento, sincronizando todo el sistema de archivos.
- F. Proceso mediante el cual se restaura el sistema o los datos tras una falla, basado en principios de consistencia y uso de logs.

Instrucciones: Asocia cada término (1-6) con la definición correspondiente (A - F). Y escríbela a continuación.

Ej: "1 - D"

